Hindi Vidya Prachar Samiti's

RAMNIRANJAN JHUNJHUNWALA COLLEGE OF ARTS, SCIENCE, AND COMMERCE

(EMPOWERED AUTONOMOUS)

Introduction to Unsupervised Learning

Name: Surajkumar Yadav

Roll no: 10302

Class: MSc. Data Science and Artificial Intelligence Part I

Ramniranjan Jhunjhunwala College of Arts, Science and Commerce

Department of Data Science and Artificial Intelligence

CERTIFICATE

This is to certify that Surajkumar Yadav of MSc. Data Science and Artificial Intelligence Roll No. 10302 has successfully completed the practical's of Introduction to Unsupervised Learning during the Academic Year 2024-2025.

Date:

(Prof. Mujtaba Shaikh)
Prof-In-Charge

External Examiner

INDEX

SR.NO	PRACTICAL NAME	DATE	SIGN
1	Implementation of data augmentation.		
2	Label Propagation using Semi-supervised learning.		
3	Perform dimensionality reduction.		
4	Implementation of Principal component analysis.		
5	Implementation of partition-based clustering.		
6	Implementation of hierarchical clustering.		
7	Implementation of density-based clustering.		
8	Perform market basket analysis using apriori algorithm.		
9	Implementation of FP- Growth algorithm.		
10	Implementation of t-SNE.		

Practical 1: Implementation of data augmentation.

[19]:
```
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.utils import array_to_img, img_to_array, load_img
```

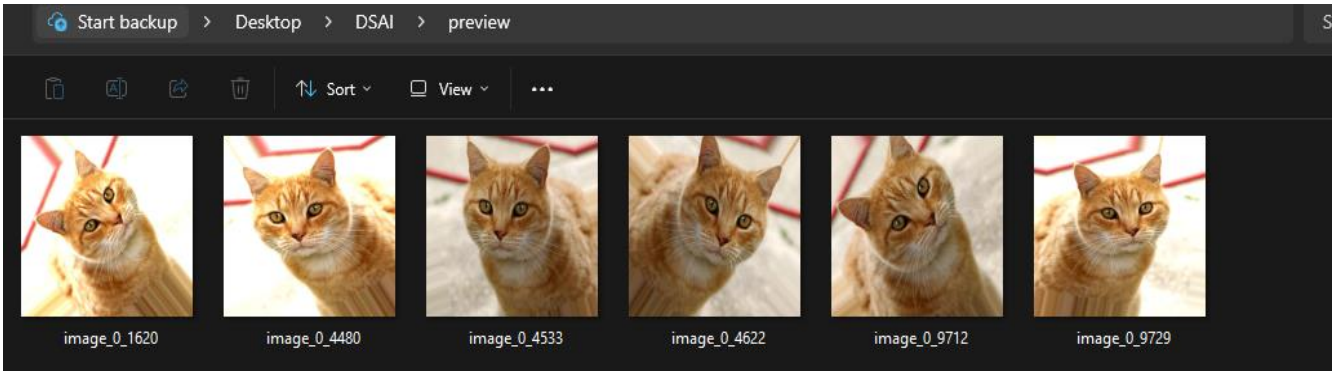
[20]:
```
data_gen = ImageDataGenerator(rotation_range = 40,
                              shear_range = 0.2,
                              zoom_range = 0.2,
                              horizontal_flip = True,
                              brightness_range = (0.5, 1.5))
```

[21]:
```
img = load_img('Cat03.jpg')
```

[22]:
```
x = img_to_array(img) # converting the input img to array
```

[23]:
```
x = x.reshape((1,) + x.shape) # reshaping the image
```

[27]:
```
i = 0

for batch in data_gen.flow(x, batch_size = 1,
                           save_to_dir = 'preview',
                           save_prefix = 'image',
                           save_format = 'jpeg'):
  i += 1
  if i > 5:
    break
```



Practical 2: Label Propagation using Semi-supervised learning

[1]:
```
!pip install scikit-learn==1.2.2
```

```
Collecting scikit-learn==1.2.2
  Downloading scikit_learn-1.2.2-cp311-cp311-win_amd64.whl.metadata (11 kB)
```

[2]:
```
import pandas as pd
import numpy as np
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from xgboost import XGBClassifier
from sklearn.metrics import accuracy_score
```

[3]:
```
X,y = make_classification(n_samples=1000, n_features=10, n_classes=4,n_informative=4,random_state=42)
```

[4]:
```
x_train ,x_test , y_train ,y_test = train_test_split(X,y,test_size=0.2,random_state=42)
```

[5]:
```
x_train.shape
```

[5]:
```
(800, 10)
```

[6]:
```
x_test.shape
```

[6]:
```
(200, 10)
```

[7]:
```
xg= XGBClassifier(objective = 'multi:softmax', num_class = 4)
```

[8]:
```
xg.fit(x_train,y_train)
```

[8]:
```
XGBClassifier(base_score=None, booster=None, callbacks=None,
              colsample_bylevel=None, colsample_bynode=None,
              colsample_bytree=None, device=None, early_stopping_rounds=None,
              enable_categorical=False, eval_metric=None, feature_types=None,
              gamma=None, grow_policy=None, importance_type=None,
              interaction_constraints=None, learning_rate=None, max_bin=None,
              max_cat_threshold=None, max_cat_to_onehot=None,
              max_delta_step=None, max_depth=None, max_leaves=None,
              min_child_weight=None, missing=nan, monotone_constraints=None,
              multi_strategy=None, n_estimators=None, n_jobs=None, num_class=4,
              num_parallel_tree=None, ...)
```

[9]:
```
y_pred = xg.predict(x_test)
```

[10]:
```
accuracy_score(y_test,y_pred)
```

[10]:
```
0.785
```

[11]:
```
x_label ,y_label=x_train[:400],y_train[:400]
x_unlabel = x_train[400:]
```

[12]:
```
xg.fit(x_label,y_label)
y_pred_label = xg.predict(x_test)
accuracy_score(y_test,y_pred_label)
```

[12]:
```
0.73
```

[13]:
```
x_label = pd.DataFrame(x_label)
y_label = pd.Series(y_label)
x_unlabel = pd.DataFrame(x_unlabel)
```

[14]:
```
while True:
  model = XGBClassifier(objective = 'multi:softmax', num_class = 4,random_state=42)
  model.fit(x_label,y_label)
  x_unlabel.reset_index(drop=True,inplace=True)
  y_pred = model.predict_proba(x_unlabel)
  index =[index for index,x in enumerate(np.max(y_pred,axis=1)) if x>0.90]
  if len(index) == 0:
    break
  temp =x_unlabel.iloc[index]
  x_unlabel.drop(index,inplace=True)
```

[15]:
```
x_unlabel
```

[15]:

	0	1	2	3	4	5	6	7	8	9
0	-0.096596	0.890467	-0.644369	2.163391	0.208599	-0.874750	0.633741	-0.426077	0.968290	0.428439
1	-1.103554	3.368361	0.497819	2.015142	0.620937	0.950295	3.098866	1.555051	-0.424070	0.043357
2	1.492592	-2.382418	1.682262	-1.490431	0.415256	-0.504839	-2.674697	-1.071531	-0.528629	-0.730062
3	-0.874510	1.185712	-2.321773	-0.335255	0.606814	-1.840432	0.584764	0.844872	-1.449066	-0.552893
4	0.909676	-1.549078	1.804284	-0.860485	-1.352823	0.975751	-1.273748	-0.633475	-1.143347	0.070923
...	...	...	...	...	...	...	...	...	...	...
193	-3.974315	1.607057	-0.601835	2.366906	1.261857	-2.435290	3.381045	-0.065062	0.130452	-0.809199
194	1.046394	0.427821	0.711641	-1.038470	0.022136	0.299620	-0.662471	0.772285	-0.512944	-0.499143
195	1.530502	1.809008	0.829943	0.709966	-1.548824	1.508748	0.344716	1.042699	0.247015	-1.156562

Practical 3: Perform dimensionality reduction

[1]:
```
import pandas as pd
import numpy as np
import seaborn as sb
import matplotlib.pyplot as plt
from sklearn.feature_selection import SelectKBest, f_classif, VarianceThreshold
from sklearn.decomposition import PCA
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
import warnings
warnings.filterwarnings('ignore')
```

[2]:
```
df = pd.read_csv("Data/first_500_rows.csv")
df.head()
```

[2]:

	Time	0	1	2	3	4	5	6	7	8	...	581
0	2008-07-19 11:55:00	3030.93	2564.00	2187.7333	1411.1265	1.3602	100.0	97.6133	0.1242	1.5005	...	NaN
1	2008-07-19 12:32:00	3095.78	2465.14	2230.4222	1463.6606	0.8294	100.0	102.3433	0.1247	1.4966	...	208.2045

[3]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 500 entries, 0 to 499
Columns: 592 entries, Time to Pass/Fail
dtypes: float64(590), int64(1), object(1)
memory usage: 2.3+ MB
```

[4]:
```
df.shape
```

[4]:
```
(500, 592)
```

[5]:
```
df.describe()
```

[5]:

	0	1	2	3	4	5	6	4
count	497.000000	498.000000	499.000000	499.000000	499.000000	499.0	499.000000	4
mean	3005.278732	2494.609378	2203.829387	1339.713650	1.275486	100.0	102.281091	
std	65.569063	90.397472	25.802867	288.301201	0.295656	0.0	3.206394	
min	2787.490000	2162.870000	2113.077800	877.626600	0.681500	100.0	93.984400	
25%	2959.650000	2453.487500	2184.005500	1064.844600	1.008100	100.0	99.875550	
50%	3007.380000	2495.740000	2199.333400	1330.671800	1.310100	100.0	102.097800	
75%	3050.290000	2539.437500	2220.711100	1588.509000	1.511900	100.0	104.172200	
max	3266.040000	2810.120000	2304.211100	2028.220800	1.884800	100.0	111.890000	

8 rows × 591 columns

[6]:
```
df.isnull().sum()
```

[6]:
```
Time          0
0             3
1             2
2             1
3             1
             ..
586           1
587           1
588           1
589           1
Pass/Fail     0
Length: 592, dtype: int64
```

[7]:
```
X = df.drop(columns = ['Time', 'Pass/Fail'])
X.head()
```

[7]:

	0	1	2	3	4	5	6	7	8	9	...	
0	3030.93	2564.00	2187.7333	1411.1265	1.3602	100.0	97.6133	0.1242	1.5005	0.0162	...	N
1	3095.78	2465.14	2230.4222	1463.6606	0.8294	100.0	102.3433	0.1247	1.4966	-0.0005	...	0.0
2	2932.61	2559.94	2186.4111	1698.0172	1.5102	100.0	95.4878	0.1241	1.4436	0.0041	...	0.0
3	2988.72	2479.90	2199.0333	909.7926	1.3204	100.0	104.2367	0.1217	1.4882	-0.0124	...	0.0
4	3032.24	2502.87	2233.3667	1326.5200	1.5334	100.0	100.3967	0.1235	1.5031	-0.0031	...	N

5 rows × 590 columns

[8]:
```
y = df['Pass/Fail']
y.head()
```

[8]:
```
0   -1
1   -1
2    1
3   -1
4   -1
Name: Pass/Fail, dtype: int64
```

Feature Selection

[9]:
```
selector = VarianceThreshold()
X_selected = selector.fit_transform(X)
print(X_selected)
```

```
[[3.0309300e+03 2.5640000e+03 2.1877333e+03 ...            nan
            nan            nan]
 [3.0957800e+03 2.4651400e+03 2.2304222e+03 ... 2.0100000e-02
  6.0000000e-03 2.0820450e+02]
 [2.9326100e+03 2.5599400e+03 2.1864111e+03 ... 4.8400000e-02
  1.4800000e-02 8.2860200e+01]
 ...
 [3.0575600e+03 2.4710100e+03 2.1807000e+03 ... 2.6000000e-02
  7.1000000e-03 1.0216520e+02]
 [2.9544100e+03 2.6149900e+03 2.2082334e+03 ... 2.2600000e-02
  7.9000000e-03 4.7408120e+02]
 [2.9972700e+03 2.6198800e+03 2.1486223e+03 ... 2.2600000e-02
  7.9000000e-03 4.7408120e+02]]
```

[10]:
```
len(X_selected)
```

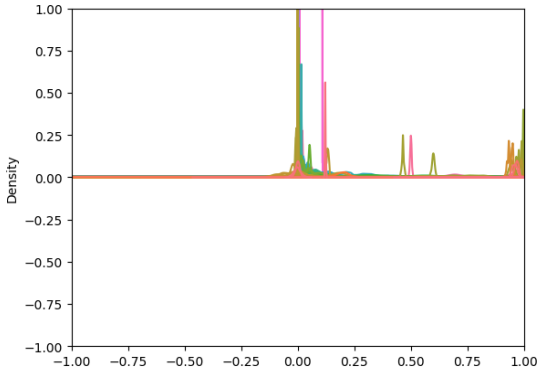
[10]:
```
500
```

[11]:
```
imputer = SimpleImputer(strategy = 'mean')
X_imputed = imputer.fit_transform(X_selected)
print(X_imputed)
```

```
[[3.03093000e+03 2.56400000e+03 2.18773330e+03 ... 1.69340681e-02
  5.45551102e-03 1.12076847e+02]
 [3.09578000e+03 2.46514000e+03 2.23042220e+03 ... 2.01000000e-02
  6.00000000e-03 2.08204500e+02]
 [2.93261000e+03 2.55994000e+03 2.18641110e+03 ... 4.84000000e-02
  1.48000000e-02 8.28602000e+01]
 ...
 [3.05756000e+03 2.47101000e+03 2.18070000e+03 ... 2.60000000e-02
  7.10000000e-03 1.02165200e+02]
 [2.95441000e+03 2.61499000e+03 2.20823340e+03 ... 2.26000000e-02
  7.90000000e-03 4.74081200e+02]
 [2.99727000e+03 2.61988000e+03 2.14862230e+03 ... 2.26000000e-02
  7.90000000e-03 4.74081200e+02]]
```

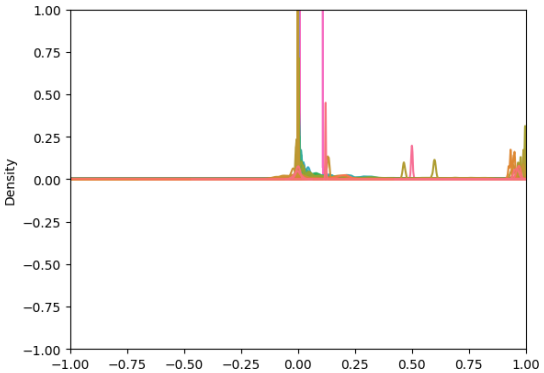
[12]:
```
sb.kdeplot(X_imputed,legend = False)
plt.xlim(-1, 1)
plt.ylim(-1, 1)
```

[12]:
```
(-1.0, 1.0)
```



[13]:
```
sb.kdeplot(X,legend = False)
plt.xlim(-1, 1)
plt.ylim(-1, 1)
```

[13]:
```
(-1.0, 1.0)
```



[14]:
```
kb = SelectKBest(score_func = f_classif, k = 20)
kb_select = kb.fit_transform(X_imputed, y)
```

[15]:
```
X_train, X_test, y_train, y_test = train_test_split(X_imputed, y, test_size = 0.2, random_state = 42)
```

Feature Extraction

[16]:
```
pca = PCA(n_components = 10)
X_train_pca = pca.fit_transform(X_train)
X_test_pca = pca.transform(X_test)
```

[17]:
```
clf_before = RandomForestClassifier(n_estimators = 100, random_state = 42)
clf_before.fit(X_train, y_train)
```

[17]:
```
RandomForestClassifier
RandomForestClassifier(random_state=42)
```

[18]:
```
y_pred_before = clf_before.predict(X_test)
acc = accuracy_score(y_test, y_pred_before)
print(f'Accuracy before dimensionality reduction:{acc}')
```

```
Accuracy before dimensionality reduction:0.86
```

[19]:
```
clf_after = RandomForestClassifier(n_estimators = 100, random_state = 42)
clf_after.fit(X_train_pca, y_train)
```

[19]:
```
RandomForestClassifier
RandomForestClassifier(random_state=42)
```

[20]:
```
y_pred_after = clf_after.predict(X_test_pca)
acc1 = accuracy_score(y_test, y_pred_after)
print(f'Accuracy after dimensionality reduction:{acc1}')
```

```
Accuracy after dimensionality reduction:0.86
```

[21]:
```
clf_before = LogisticRegression()
clf_before.fit(X_train, y_train)
```

[21]:
```
LogisticRegression
LogisticRegression()
```

[22]:
```
y_pred_before = clf_before.predict(X_test)
acc = accuracy_score(y_test, y_pred_before)
print(f'Accuracy before dimensionality reduction:{acc}')
```

```
Accuracy before dimensionality reduction:0.85
```

[23]:
```
clf_after = LogisticRegression()
clf_after.fit(X_train_pca, y_train)
```

[23]:
```
LogisticRegression
LogisticRegression()
```

[24]:
```
y_pred_after = clf_after.predict(X_test_pca)
acc1 = accuracy_score(y_test, y_pred_after)
print(f'Accuracy after dimensionality reduction:{acc1}')
```

```
Accuracy after dimensionality reduction:0.86
```

[25]:
```
print(pca.explained_variance_ratio_)
print(f"Total explained variance: {sum(pca.explained_variance_ratio_)}")
```

```
[0.51838776 0.29496934 0.06091871 0.05093539 0.02353126 0.01715892
 0.00553501 0.00466569 0.00315293 0.00246198]
Total explained variance: 0.9817169765299132
```

- PCA retains most of the important information in the dataset by focusing on components with the highest variance.
- If the first 10 principal components (as per n_components=10) capture nearly all the variance of the original dataset, the information loss due to dimensionality reduction will be negligible.
- The classifier will then have access to nearly the same "quality" of data, resulting in little or no change in accuracy.

Practical 4: Implementation of Principal component analysis

[1]:
```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
from mpl_toolkits.mplot3d import Axes3D
import plotly.express as exp
```

[2]:
```
df = pd.read_csv('Data/mnist_train.csv')
df.head()
```

[2]:

	label	1x1	1x2	1x3	1x4	1x5	1x6	1x7	1x8	1x9	...	28x19	28x20	28x21	28x22	28x23	28x24
0	5	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0
2	4	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0
3	1	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0
4	9	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0

5 rows × 785 columns

[5]:
```
x = df.drop('label', axis = 1)
x.head()
```

[5]:

	1x1	1x2	1x3	1x4	1x5	1x6	1x7	1x8	1x9	1x10	...	28x19	28x20	28x21	28x22	28x23
0	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0
2	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0
3	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0
4	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0

5 rows × 784 columns

[6]:
```
y = df['label']
y.head()
```

[6]:
```
0    7
1    2
2    1
3    0
4    4
Name: label, dtype: int64
```

PCA

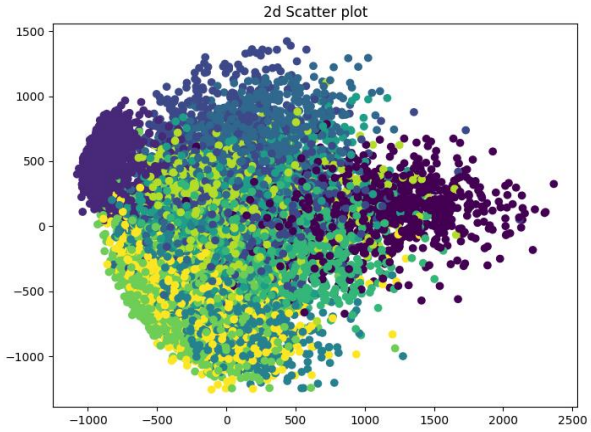
[7]:
```
pca = PCA(n_components = 2)
X_two = pca.fit_transform(x)
print(X_two)
```

```
[[-411.26219926 -686.57150042]
 [  58.05972848  983.11999352]
 [-935.10439332  459.08319218]
 ...
 [-282.41253522 -550.82773027]
 [-287.26973663  155.86885152]
 [1144.16758623   22.69576656]]
```

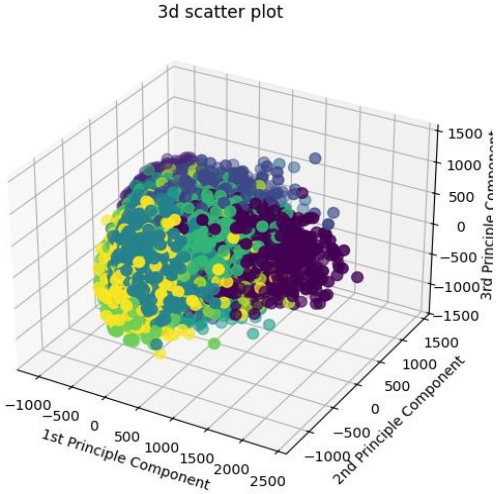
[8]:
```
pca = PCA(n_components = 3)
X_three = pca.fit_transform(x)
print(X_three)
```

```
[[-411.26169251 -686.55865199  -51.1756136 ]
 [  58.06457709  983.17614762    8.78673593]
 [-935.10434569  459.07120944  320.10990867]
 ...
 [-282.41490609 -550.85126022  197.56105651]
 [-287.27129233  155.86691696  536.79630436]
 [1144.17004782   22.71286375  849.57168084]]
```

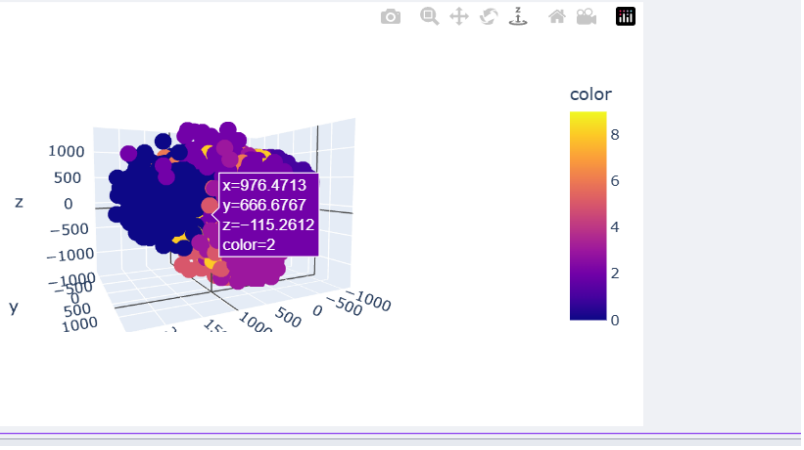
[9]:
```
plt.figure(figsize = [8,6])
plt.title("2d Scatter plot")
plt.scatter(X_two[:,0], X_two[:, 1], c=y)
plt.show()
```



[10]:
```
fig = plt.figure(figsize = [8,6])
ax = fig.add_subplot(111, projection='3d')
ax.scatter(X_three[:, 0], X_three[:, 1], X_three[:, 2], c=y, cmap='viridis', s=60)
ax.set_title("3d scatter plot")
ax.set_xlabel("1st Principle Component")
ax.set_ylabel("2nd Principle Component")
ax.set_zlabel("3rd Principle Component")
plt.show()
```



[11]:
```
fig = exp.scatter_3d(x = X_three[:, 0], y = X_three[:, 1], z = X_three[:, 2], color=y)
fig.show()
```



PCA 2

[1]:
```
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler
```

[2]:
```
df = pd.DataFrame({
    "X":[2.5, 0.5, 2.2, 1.9, 3.1, 2.3, 2, 1, 1.5, 1.1],
    "Y":[2.4, 0.7, 2.9, 2.2, 3, 2.7, 1.6, 1.1, 1.6, 0.9]})
df.head()
```

[2]:

	X	Y
0	2.5	2.4
1	0.5	0.7
2	2.2	2.9
3	1.9	2.2
4	3.1	3.0

[3]:
```
scaler = StandardScaler()
```

[4]:
```
df[["X", "Y"]] = scaler.fit_transform(df[["X", "Y"]])
df.head()
```

[4]:

	X	Y
0	0.926279	0.610169
1	-1.758587	-1.506743
2	0.523549	1.232790
3	0.120819	0.361120

[5]:
```
X = df["X"]
Y = df["Y"]
```

[6]:
```
cov_mat = np.cov(X,Y)
cov_mat
```

[6]:
```
array([[1.11111111, 1.0288103 ],
       [1.0288103 , 1.11111111]])
```

[7]:
```
df.head()
```

[7]:

	X	Y
0	0.926279	0.610169
1	-1.758587	-1.506743
2	0.523549	1.232790
3	0.120819	0.361120
4	1.731739	1.357314

[8]:
```
eigen_values, eigen_vectors = np.linalg.eig(cov_mat)
```

[9]:
```
print("Eigen Vectors:", eigen_vectors)
```

```
Eigen Vectors: [[ 0.70710678 -0.70710678]
 [ 0.70710678  0.70710678]]
```

[10]:
```
print("Eigen Values:",eigen_values)
```

```
Eigen Values: [2.13992141 0.08230081]
```

[11]:
```
transformed_df = np.dot(df.iloc[:,0:2], eigen_vectors)
new_df = pd.DataFrame(transformed_df, columns = ['PC1', 'PC2'])
#new_df['target'] = df['target'].values
new_df.head()
```

[11]:

	PC1	PC2
0	1.086432	-0.223524
1	-2.308937	0.178081
2	1.241919	0.501509
3	0.340782	0.169919
4	2.184290	-0.264758

Practical 5: Implementation of partition-based clustering.

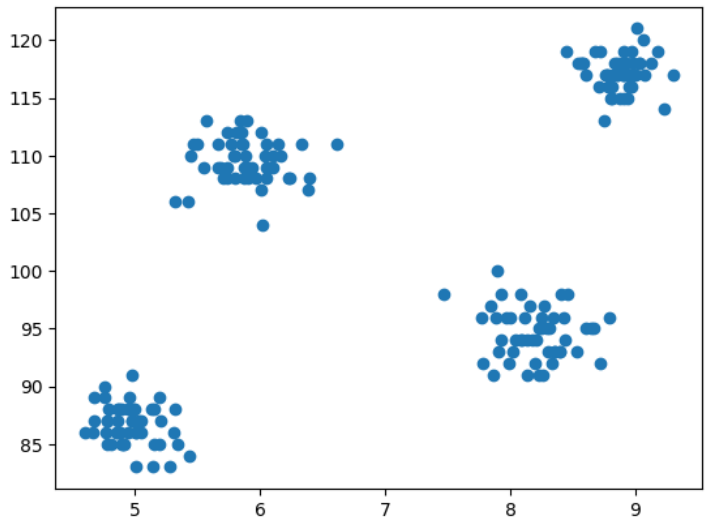
[1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

[2]:
```
df=pd.read_csv('Data/student_clustering.csv')
df.head()
```

[2]:

	cgpa	ML
0	5.13	88
1	5.90	113
2	8.36	93
3	8.27	97
4	5.45	110

[3]:
```
plt.scatter(df['cgpa'],df['ML'])
```

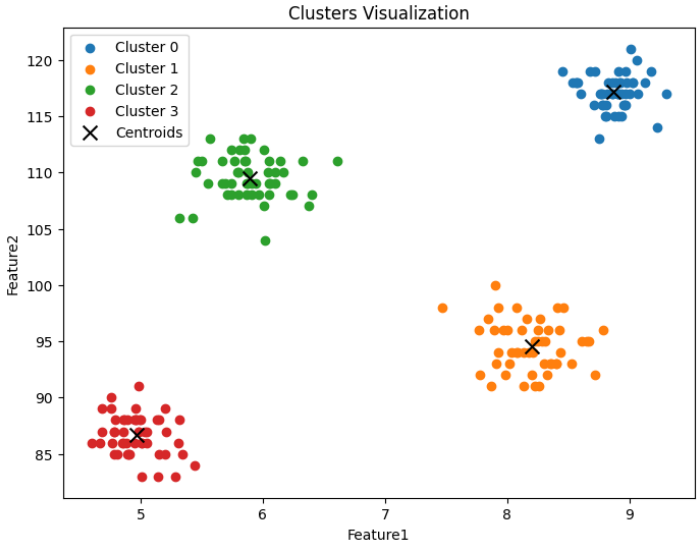


[13]:

```python
def euclidean_distance(point1, point2):
    return np.sqrt(np.sum((point1 - point2) ** 2))

def kmeans_clustering(data, k,max_iter=500):
    centroids = data[np.random.choice(len(data), k, replace=False)]

    while True:
        # Compute distances and assign clusters
        distances = np.array([[euclidean_distance(point, centroid) for centroid in centroids] for point in data])
        clusters = np.argmin(distances, axis=1)

        # Calculate new centroids
        new_centroids = np.array([
            data[clusters == i].mean(axis=0) if np.any(clusters == i) else centroids[i] for i in range(k)
        ])
        # Checking the centriod in diff iterations
        if np.allclose(centroids, new_centroids):
            break
        iter_count = 0
        centroids = new_centroids
        iter_count += 1
        if iter_count >= max_iter:
            break

    return clusters, centroids
```

```python
data = df.to_numpy()   # Convert df to array
k = int(input("Enter number of cluster (k) : "))

clusters, centroids = kmeans_clustering(data, k)

df['Cluster'] = clusters

print(df)

plt.figure(figsize=(8, 6))
for i in range(k):
    cluster_data = df[df['Cluster'] == i]
    plt.scatter(cluster_data['cgpa'], cluster_data['ML'], label=f'Cluster {i}')

plt.scatter(centroids[:, 0], centroids[:, 1], color='black', marker='x', s=100, label='Centroids')

plt.title('Clusters Visualization')
plt.xlabel('Feature1')
plt.ylabel('Feature2')
plt.legend()
plt.show()
```

```
Enter number of cluster (k) :  4
     cgpa   ML  Cluster  cluster
0    5.13   88        3        2
1    5.90  113        2        3
2    8.36   93        1        0
3    8.27   97        1        0
4    5.45  110        2        3
..    ...  ...      ...      ...
195  4.68   89        3        2
196  8.57  118        0        1
197  5.85  112        2        3
198  6.23  108        2        3
199  8.82  117        0        1

[200 rows x 4 columns]
```



Using Libraries

[6]:
```
from sklearn.cluster import KMeans
kmeans = KMeans(n_clusters = 4)
kmeans.fit(df)
```

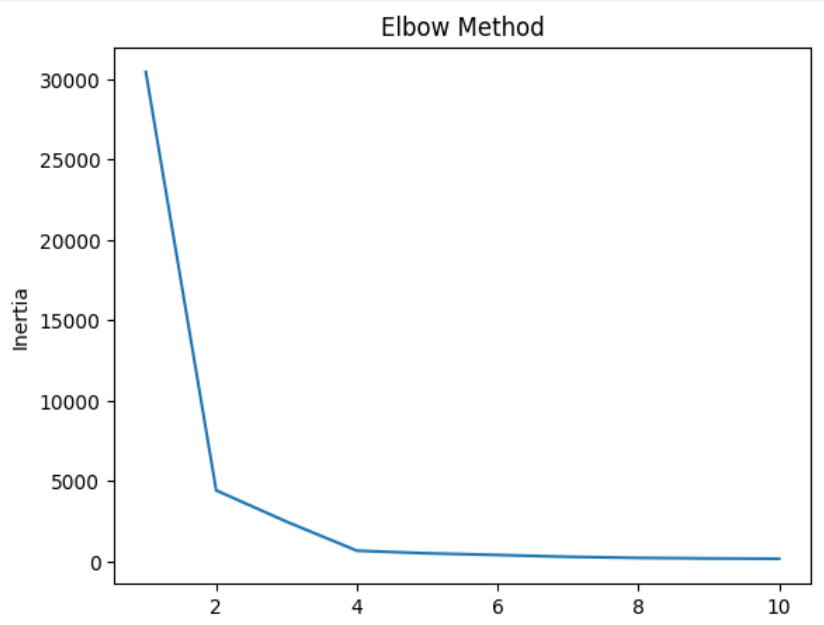
[6]:
```
KMeans(n_clusters=4)
```

[7]:
```
df['cluster'] = kmeans.labels_
```

[8]:
```
plt.scatter(df['cgpa'], df['ML'], c=df['cluster'], cmap='rainbow')
plt.xlabel('CGPA')
plt.ylabel('ML Score')
plt.title('K-Means Clustering')
plt.show()
```

[27]:
```
wcss = []
for i in range(1, 11):
    kmeans = KMeans(n_clusters=i)
    label=kmeans.fit_predict(df)
    wcss.append(kmeans.inertia_)
print(wcss)

plt.plot(range(1, 11), wcss)
plt.xlabel('Number of Clusters')
plt.ylabel('Inertia')
plt.title('Elbow Method')
plt.show()
```

```
[30457.898288, 4434.14127, 2487.7133489999997, 681.9696599999999, 518.9693726248037, 416.46718557718464, 300.23918959431916, 233.
8704953466, 183.62520804948647]
```



[26]:
```
# Silhouette score
print(f'Silhoutee Score: {(silhouette_score(df,label))}')
```

```
Silhoutee Score: 0.560302034088238
```

Practical 6: Implementation of hierarchical clustering

[1]:
```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
import scipy.cluster.hierarchy as sch
from sklearn.cluster import AgglomerativeClustering
```

[2]:
```
data = load_iris()
```

[3]:
```
X = data.data
```

[4]:
```
data.target_names
```

[4]:
```
array(['setosa', 'versicolor', 'virginica'], dtype='<U10')
```

[5]:
```
data.feature_names
```

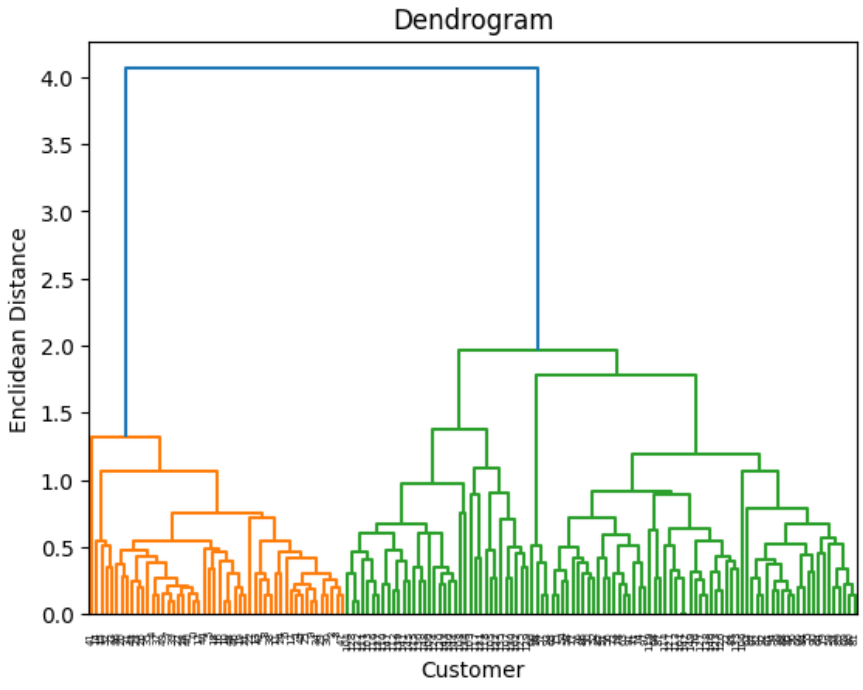
[5]:
```
['sepal length (cm)',
 'sepal width (cm)',
 'petal length (cm)',
 'petal width (cm)']
```

[6]:
```
df = pd.DataFrame(X, columns = data.feature_names)
df.head()
```

[6]:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2

[7]:
```
dendrogram = sch.dendrogram(sch.linkage(df, method = 'average'))

plt.title('Dendrogram')
plt.xlabel('Customer')
plt.ylabel('Enclidean Distance')
plt.show()
```



[8]:
```
ag = AgglomerativeClustering(n_clusters = 3)
ag.fit(df)
```

[8]:
```
AgglomerativeClustering(n_clusters=3)
```

[9]:
```
pred = ag.labels_
pred
```

[9]:
```
array([1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 2, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 2, 0, 2, 2, 2, 2, 0, 2, 2, 2,
       2, 2, 2, 0, 0, 2, 2, 2, 2, 0, 2, 0, 2, 0, 2, 2, 0, 0, 2, 2, 2, 2,
       2, 0, 0, 2, 2, 2, 0, 2, 2, 2, 0, 2, 2, 2, 0, 2, 2, 0], dtype=int64)
```

[10]:
```
data.target
```

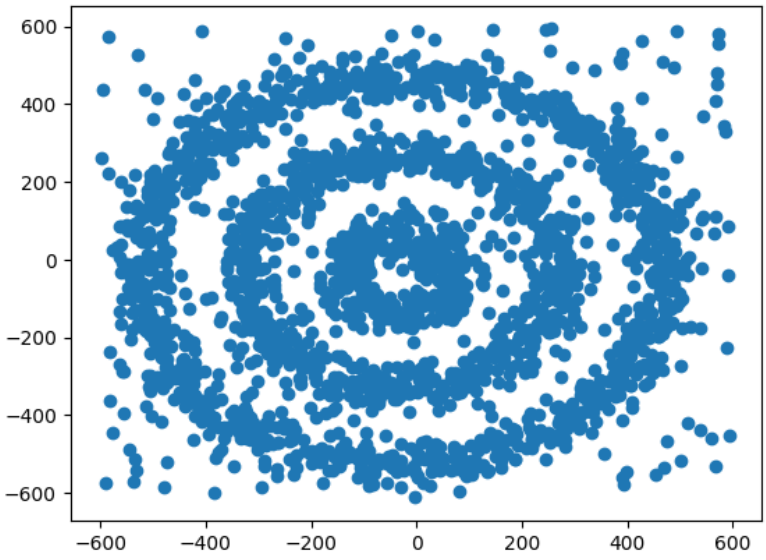
[10]:
```
array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
       2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
       2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2])
```

Practical 7: Implementation of density-based clustering

[1]:
```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
import warnings
warnings.filterwarnings('ignore')
```

[2]:
```
df=pd.read_csv('data/DBSCAN.csv')
```

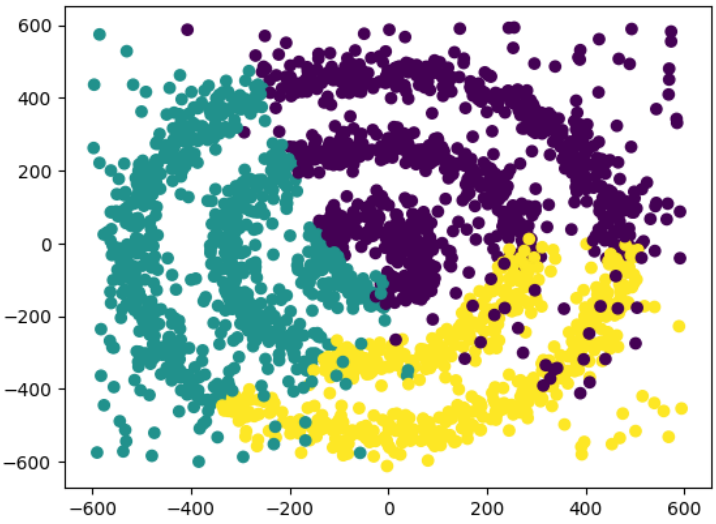
[3]:
```
plt.scatter(df['A'],df['B'])
```



[4]:
```
kmeans = KMeans(n_clusters = 3)
kmeans.fit(df)
```

[4]:
```
KMeans
KMeans(n_clusters=3)
```

[5]:
```
labels = kmeans.labels_
plt.scatter(df['A'], df['B'], c=labels)
plt.show()
```

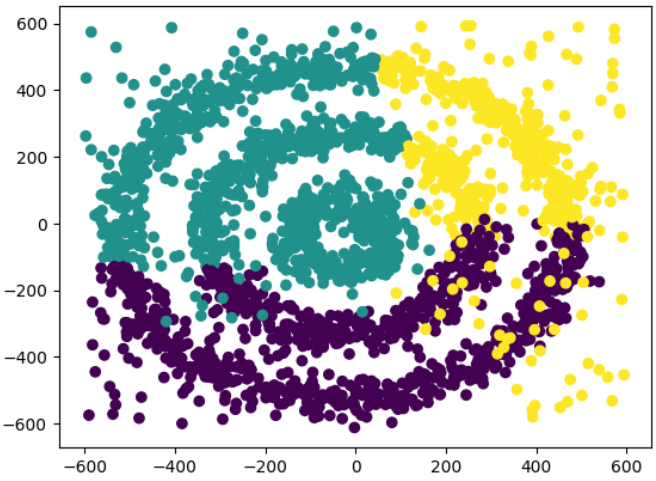


[6]:
```
from sklearn.cluster import AgglomerativeClustering
ac = AgglomerativeClustering( n_clusters=3).fit(df)
ac
```

[6]:
```
AgglomerativeClustering(n_clusters=3)
```

[7]:
```
labels= ac.labels_
```

[8]:
```
plt.scatter(df['A'], df['B'], c=labels)
plt.show()
```



[9]:
```
from sklearn.cluster import DBSCAN
```

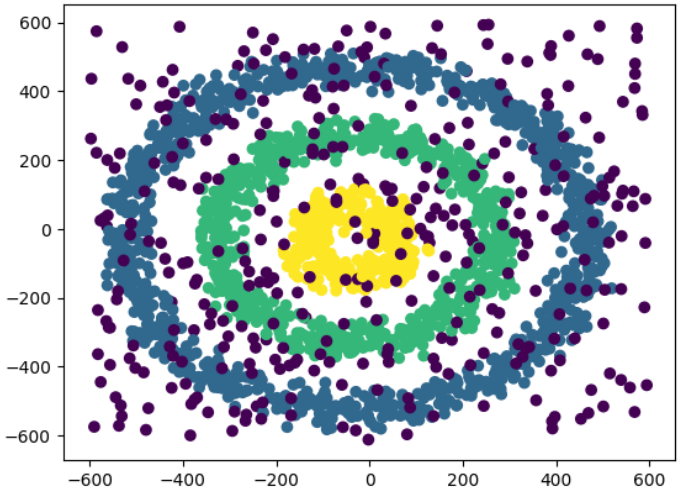
[10]:
```
db=DBSCAN(eps=40,min_samples=10)
db.fit(df)
```

[10]:
```
DBSCAN
DBSCAN(eps=40, min_samples=10)
```

[11]:
```
df['labels']=db.labels_
df['labels'].unique()
```

[11]:
```
array([ 0, -1,  1,  2], dtype=int64)
```

[12]:
```
plt.scatter(df['A'], df['B'], c=df['labels'])
plt.show()
```



Practical 8: Perform market basket analysis using apriori algorithm.

[1]:
```
!pip install mlxtend==0.23.1
```

```
Collecting mlxtend==0.23.1 •••
```

[19]:
```
import pandas as pd
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori, association_rules
```

Creating a Customize dataset

[25]:
```
data = [['milk', 'onion', 'nutmeg', 'kidney beans', 'eggs', 'yogurt'],
        ['chips', 'bananas', 'nutmeg', 'eggs', 'curd', 'red bull'],
        ['pani puri', 'chips', 'maggie', 'eggs', 'peanuts', 'onion'],
        ['chicken', 'fish', 'bread','banana'],
        ['chicken', 'milk', 'yogurt', 'bread', 'chips'],
        ['rice', 'apple', 'chips']]
data
```

[25]:
```
[['milk', 'onion', 'nutmeg', 'kidney beans', 'eggs', 'yogurt'],
 ['chips', 'bananas', 'nutmeg', 'eggs', 'curd', 'red bull'],
 ['pani puri', 'chips', 'maggie', 'eggs', 'peanuts', 'onion'],
 ['chicken', 'fish', 'bread', 'banana'],
 ['chicken', 'milk', 'yogurt', 'bread', 'chips'],
 ['rice', 'apple', 'chips']]
```

NaN=Null can have value 0 but None is nothing

[26]:
```
te = TransactionEncoder()
```

[27]:
```
te_array = te.fit(data).transform(data)
te_array
```

[27]:
```
array([[False, False, False, False, False, False, False,  True, False,
         True, False,  True,  True,  True, False, False, False, False,
         True],
       [False, False,  True, False, False,  True,  True,  True, False,
        False, False, False,  True, False, False, False,  True, False,
        False],
       [False, False, False, False, False,  True, False,  True, False,
        False,  True, False, False,  True,  True,  True, False, False,
        False],
       [False,  True, False,  True,  True, False, False, False,  True,
        False, False, False, False, False, False, False, False, False,
        False],
       [False, False, False,  True,  True,  True, False, False, False,
        False, False,  True, False, False, False, False, False, False,
         True],
       [ True, False, False, False, False,  True, False, False, False,
        False, False, False, False, False, False, False, False,  True,
        False]])
```

[28]:
```
df = pd.DataFrame(te_array, columns = te.columns_)
df.head()
```

[28]:

	apple	banana	bananas	bread	chicken	chips	curd	eggs	fish	kidney beans	maggie	milk	nutmeg	onion	pani puri	peanuts	red bull	ri
0	False	False	False	False	False	False	False	True	False	True	False	True	True	True	False	False	False	Fal
1	False	False	True	False	False	True	True	True	False	False	False	False	True	False	False	False	True	Fal

[29]:
```
fre_items = apriori(df, min_support = 0.3, use_colnames = True)
fre_items
```

[29]:

	support	itemsets
0	0.333333	(bread)
1	0.333333	(chicken)
2	0.666667	(chips)
3	0.500000	(eggs)
4	0.333333	(milk)
5	0.333333	(nutmeg)
6	0.333333	(onion)
7	0.333333	(yogurt)

[30]:
```
arm = association_rules(fre_items, metric = 'confidence', min_threshold = 0.2)
arm[['antecedents', 'consequents', 'support',   'confidence']]
```

[30]:

	antecedents	consequents	support	confidence
0	(bread)	(chicken)	0.333333	1.000000
1	(chicken)	(bread)	0.333333	1.000000
2	(eggs)	(chips)	0.333333	0.666667
3	(chips)	(eggs)	0.333333	0.500000
4	(nutmeg)	(eggs)	0.333333	1.000000
5	(eggs)	(nutmeg)	0.333333	0.666667
6	(eggs)	(onion)	0.333333	0.666667
7	(onion)	(eggs)	0.333333	1.000000
8	(milk)	(yogurt)	0.333333	1.000000
9	(yogurt)	(milk)	0.333333	1.000000

Practical 9: Implementation of FP- Growth algorithm

[1]:
```
!pip install mlxtend==0.23.1
```

```
Requirement already satisfied: mlxtend==0.23.1 in c:\users\uday8\appdata\local\programs\python\python311\lib\site-pa
Requirement already satisfied: scipy>=1.2.1 in c:\users\uday8\appdata\local\programs\python\python311\lib\site-packa
```

[18]:
```
import pandas as pd
import numpy as np
```

[19]:
```
data = [
    ['f','a','c','d','g','i','m','p'],
   ['a','b','c','f','l','m','o'],
    ['b','f','h','j','o'],
    ['b','c','k','s','p'],
   ['a','f','c','e','l','p','m','n']
]
```

[20]:
```
data
```

[20]:
```
[['f', 'a', 'c', 'd', 'g', 'i', 'm', 'p'],
 ['a', 'b', 'c', 'f', 'l', 'm', 'o'],
 ['b', 'f', 'h', 'j', 'o'],
 ['b', 'c', 'k', 's', 'p'],
 ['a', 'f', 'c', 'e', 'l', 'p', 'm', 'n']]
```

[21]:
```
df = pd.DataFrame(data)
```

[22]:
```
df
```

[22]:

	0	1	2	3	4	5	6	7
0	f	a	c	d	g	i	m	p
1	a	b	c	f	l	m	o	None
2	b	f	h	j	o	None	None	None
3	b	c	k	s	p	None	None	None
4	a	f	c	e	l	p	m	n

[7]:
```
df.nunique()
```

[7]:
```
0    3
1    4
2    3
3    5
4    4
5    3
6    2
7    2
dtype: int64
```

[9]:
```
from mlxtend.preprocessing import TransactionEncoder

te=TransactionEncoder()
te_ar = te.fit(data).transform(data)
te_ar
```

[9]:
```
array([[ True, False,  True,  True, False,  True,  True, False,  True,
        False, False, False,  True, False, False,  True, False],
       [ True,  True,  True, False, False,  True, False, False, False,
        False, False,  True,  True, False,  True, False, False],
       [False,  True, False, False, False,  True, False,  True, False,
         True, False, False, False, False,  True, False, False],
       [False,  True,  True, False, False, False, False, False, False,
        False,  True, False, False, False, False,  True,  True],
       [ True, False,  True, False,  True,  True, False, False, False,
        False, False,  True,  True,  True, False,  True, False]])
```

[10]:
```
dataset= pd.DataFrame(te_ar,columns=te.columns_)
dataset
```

[10]:

	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	s
0	True	False	True	True	False	True	True	False	True	False	False	False	True	False	False	True	False
1	True	True	True	False	False	True	False	False	False	False	False	True	True	False	True	False	False
2	False	True	False	False	False	True	False	True	False	True	False	False	False	False	True	False	False

[11]:
```
from mlxtend.frequent_patterns import fpgrowth
res =fpgrowth(dataset,min_support=0.6,use_colnames=True,max_len=1)
```

[12]:
```
res
```

[12]:

	support	itemsets
0	0.8	(f)
1	0.8	(c)
2	0.6	(p)
3	0.6	(m)
4	0.6	(a)
5	0.6	(b)

[13]:
```
from mlxtend.frequent_patterns import association_rules
res1 = association_rules(res,metric='confidence',min_threshold=0.6)
```

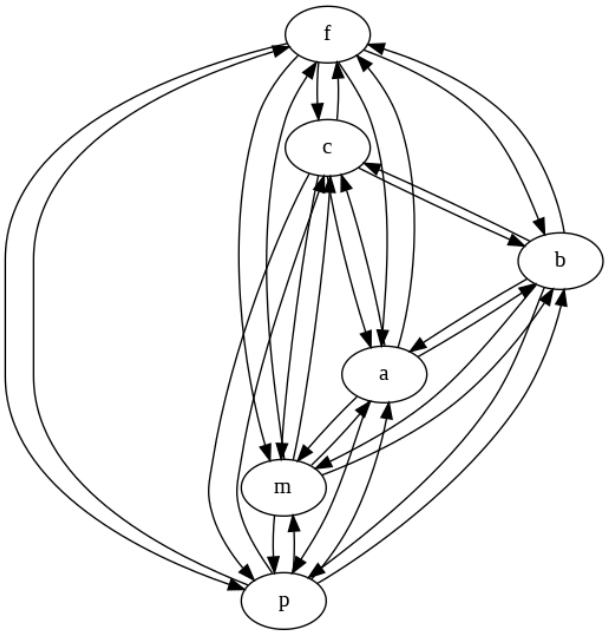
[14]:
```
res1[['antecedents','consequents','support','confidence']].sort_values(by='confidence',ascending=False)
```

[14]:

	antecedents	consequents	support	confidence

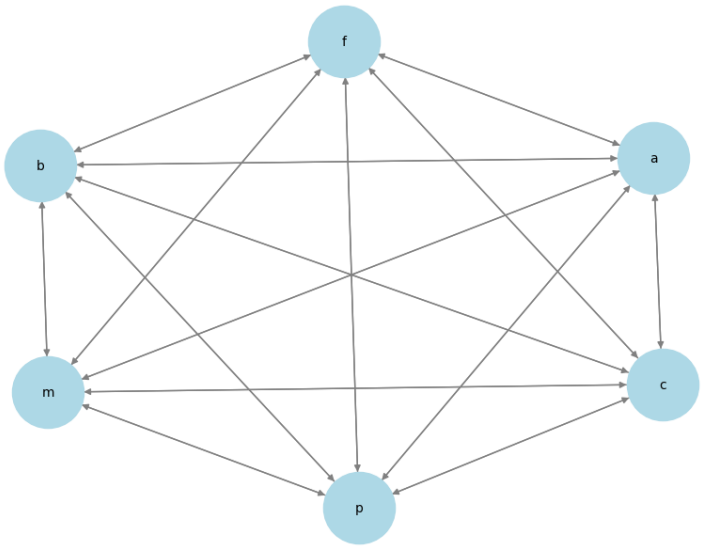
[16]:

```
import graphviz
graph = graphviz.Digraph('G', format='png')

# Add nodes and edges based on frequent itemsets
for index, row in res.iterrows():
    itemset = ', '.join(row['itemsets'])
    graph.node(itemset)
    for connected_itemset in res['itemsets']:
        if row['itemsets'] != connected_itemset:
            graph.edge(itemset, ', '.join(connected_itemset))

# Render the graph (this saves the file to disk)
graph.render('frequent_itemsets_tree')
```



[23]:

```
import networkx as nx
import matplotlib.pyplot as plt

# Convert frozensets to string labels for visualization
res['itemsets'] = res['itemsets'].apply(lambda x: ', '.join(sorted(x)))

# Create a directed graph
G = nx.DiGraph()

# Add nodes
for itemset in res['itemsets']:
    G.add_node(itemset)

# Add edges based on itemset relationships
for index, row in res.iterrows():
    for item in res['itemsets']:
        if row['itemsets'] != item:
            G.add_edge(row['itemsets'], item)

# Draw the graph
plt.figure(figsize=(8, 6))
pos = nx.spring_layout(G, seed=42)
nx.draw(G, pos, with_labels=True, node_size=3000,
        node_color="lightblue", font_size=10, edge_color="gray")
plt.show()
```



Practical 10: Implementation of t-SNE

[50]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
from sklearn.manifold import TSNE
```

[51]:
```
df= pd.read_csv('Data/human_liver.tsv',sep='\t')
```

[52]:
```
df
```

[52]:

	genes	GSM742944	GSM2326089	GSM1807974	GSM1807990	GSM2055788	GSM2142335	GSM1807979	GSM1695909	GSM1
0	A1BG	0	92	11663	2089	18808	183	5123	79640	

[53]:
```
df.describe()
```

[53]:

	GSM742944	GSM2326089	GSM1807974	GSM1807990	GSM2055788	GSM2142335	GSM1807979	GSM1695909
count	35238.000000	35238.000000	35238.000000	35238.000000	3.523800e+04	3.523800e+04	35238.000000	3.523800e+04
mean	9.136245	1499.294171	473.682473	82.726971	4.680936e+02	5.455975e+02	280.928174	1.572038e+03
std	211.284156	8817.639459	3415.314857	1193.182428	7.613447e+03	1.129007e+04	2608.339591	2.874910e+04

[54]:
```
df.columns
```

[54]:
```
Index(['genes', 'GSM742944', 'GSM2326089', 'GSM1807974', 'GSM1807990',
       'GSM2055788', 'GSM2142335', 'GSM1807979', 'GSM1695909', 'GSM1554468',
       ...
       'GSM2653842', 'GSM2653843', 'GSM2653844', 'GSM2653845', 'GSM2653846',
       'GSM2653847', 'GSM2653849', 'GSM2653850', 'GSM2653851', 'GSM2653853'],
      dtype='object', length=904)
```

[55]:
```
df.shape
```

[55]:
```
(35238, 904)
```

[56]:
```
df1= df.drop('genes',axis=1)
```

[57]:
```
df1
```

[57]:

	GSM742944	GSM2326089	GSM1807974	GSM1807990	GSM2055788	GSM2142335	GSM1807979	GSM1695909	GSM1554468
0	0	92	11663	2089	18808	183	5123	79640	1595

[58]:
```
df1 = df1.values.transpose()
df1.shape
```

[58]:
```
(903, 35238)
```

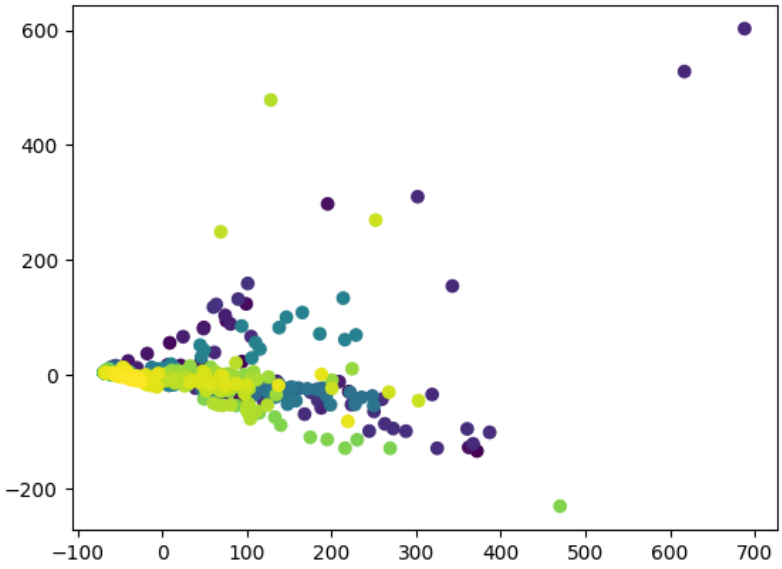
[59]:
```
# Standradize
sc= StandardScaler()
df1= sc.fit_transform(df1)
```

[60]:
```
pca = PCA(n_components=2)
x_pca= pca.fit_transform(df1.copy())
```

[61]:
```
x_pca
```

[61]:
```
array([[ -67.9080145 ,    2.88905558],
       [ 371.7820751 , -133.46789707],
       [  38.35319593,  -11.91123058],
       ...,
       [ -36.30488889,   -8.15713059],
       [ -35.80987968,   -9.35225669],
       [ -45.61920125,   -4.12472877]])
```

[70]:
```
# scatter plot with respect to rows amd color
plt.scatter(x=x_pca[:,0],y=x_pca[:,1],c=range(903), cmap='viridis')
```



[71]:
```
# t-SNE
tsne= TSNE(n_components=2)
x_tsne= tsne.fit_transform(df1)
```

[72]:
```
x_tsne
```

[72]:
```
array([[-6.157799 , -2.5120385],
       [23.322325 ,  6.306721 ],
       [10.936506 ,  1.0523735],
       ...,
       [-1.0163583,  7.694868 ],
       [-1.016027 ,  7.696485 ],
       [-2.2351305,  6.6415277]], dtype=float32)
```

[73]:
```
plt.scatter(x=x_tsne[:,0],y=x_tsne[:,1],c=range(903), cmap='viridis')
plt.show()
```